

# Solucionario de ejercicios C++ para Code::Blocks

Ejercicios 1.1 al 6.2 | Fondo blanco y código con palabras reservadas resaltadas

## Ejercicio 1.1 — Contador de muertes en Fortnite

Código completo y revisado para compilar en Code::Blocks:

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

int main() {
    // ENTRADA
    string nombreJugador;
    int kills, deaths;
    double danioTotal;

    cout << "Ingrese nombre del jugador: ";
    getline(cin, nombreJugador);

    cout << "Ingrese kills: ";
    cin >> kills;

    cout << "Ingrese deaths: ";
    cin >> deaths;

    cout << "Ingrese danio total infligido: ";
    cin >> danioTotal;

    // PROCESO
    double asistencias = kills * 0.3;
    double kda;

    if (deaths > 0) {
        kda = (kills + asistencias) / deaths;
    } else {
        kda = kills + asistencias;
    }

    double danioPromedio = 0;

    if (kills > 0) {
        danioPromedio = danioTotal / kills;
    }

    bool gano = (kills >= 5 && deaths <= 2);

    // SALIDA
    cout << fixed << setprecision(2);
    cout << "\n==== RESUMEN DE PARTIDA =====> << endl;
    cout << "Jugador: " << nombreJugador << endl;
    cout << "Kills: " << kills << endl;
    cout << "Deaths: " << deaths << endl;
    cout << "Danio total: " << danioTotal << endl;
    cout << "Asistencias estimadas: " << asistencias << endl;
    cout << "KDA: " << kda << endl;

    if (kills > 0) {
        cout << "Danio promedio por kill: " << danioPromedio << endl;
    } else {
        cout << "Sin kills esta partida." << endl;
    }

    if (gano) {
        cout << "Resultado: El jugador gano la partida." << endl;
    } else {
        cout << "Resultado: El jugador no gano la partida." << endl;
    }

    return 0;
}
```

## Ejercicio 1.2 — Calculadora de calorías Fitness

Código completo y revisado para compilar en Code::Blocks:

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

int main() {
    // ENTRADA
    string nombreUsuario;
    double peso;
    int tipoActividad;
    double minutos;

    cout << "Ingrese nombre del usuario: ";
    getline(cin, nombreUsuario);

    cout << "Ingrese peso en kg: ";
    cin >> peso;

    cout << "Ingrese tipo de actividad:" << endl;
    cout << "1. Caminar" << endl;
    cout << "2. Correr" << endl;
    cout << "3. Ciclismo" << endl;
    cout << "4. Natacion" << endl;
    cout << "Opcion: ";
    cin >> tipoActividad;

    cout << "Ingrese minutos practicados: ";
    cin >> minutos;

    // PROCESO
    double met;

    if (tipoActividad == 1) {
        met = 3.5;
    } else if (tipoActividad == 2) {
        met = 9.0;
    } else if (tipoActividad == 3) {
        met = 7.5;
    } else if (tipoActividad == 4) {
        met = 8.0;
    } else {
        met = 3.5;
    }

    double calorias = (met * peso * minutos) / 60.0;
    string clasificacion;

    if (calorias < 200) {
        clasificacion = "Esfuerzo leve";
    } else if (calorias <= 400) {
        clasificacion = "Buen entrenamiento";
    } else {
        clasificacion = "Quema total";
    }

    // SALIDA
    cout << fixed << setprecision(2);
    cout << "\n==== RESUMEN FITNESS =====" << endl;
    cout << "Usuario: " << nombreUsuario << endl;
    cout << "Peso: " << peso << " kg" << endl;
    cout << "Minutos: " << minutos << endl;
    cout << "MET usado: " << met << endl;
    cout << "Calorias quemadas: " << calorias << endl;
    cout << "Clasificacion: " << clasificacion << endl;

    return 0;
}
```

## Ejercicio 2.1 — Clasificador de películas Netflix

Código completo y revisado para compilar en Code::Blocks:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    // ENTRADA
    string nombreUsuario;
    int edad;
    string clasificacion;
    string pelicula;

    cout << "Ingrese nombre del usuario: ";
    getline(cin, nombreUsuario);

    cout << "Ingrese edad: ";
    cin >> edad;

    cin.ignore();

    cout << "Ingrese nombre de la pelicula: ";
    getline(cin, pelicula);

    cout << "Ingrese clasificacion (G, PG, PG13, R, NC17): ";
    getline(cin, clasificacion);

    // PROCESO Y SALIDA
    cout << "\n==== RESULTADO NETFLIX =====> << endl;
    cout << "Usuario: " << nombreUsuario << endl;
    cout << "Pelicula: " << pelicula << endl;
    cout << "Clasificacion: " << clasificacion << endl;

    if (clasificacion == "G" || clasificacion == "PG") {
        cout << "Disfruta " << pelicula << "!" << endl;
    } else if (clasificacion == "PG13") {
        if (edad >= 13) {
            cout << "Disfruta " << pelicula << "!" << endl;
        } else {
            cout << "Contenido restringido. Necesitas " << (13 - edad) << " años mas." << endl;
        }
    } else if (clasificacion == "R") {
        if (edad >= 17) {
            cout << "Disfruta " << pelicula << "!" << endl;
        } else {
            cout << "Contenido restringido. Necesitas " << (17 - edad) << " años mas." << endl;
        }
    } else if (clasificacion == "NC17") {
        if (edad >= 18) {
            cout << "Disfruta " << pelicula << "!" << endl;
        } else {
            cout << "Contenido restringido. Necesitas " << (18 - edad) << " años mas." << endl;
        }
    } else {
        cout << "Clasificacion no reconocida." << endl;
    }

    return 0;
}
```

## Ejercicio 2.2 — Descuento en tienda ZARA

*Código completo y revisado para compilar en Code::Blocks:*

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    // ENTRADA
    double montoCompra;
    int diaSemana;

    cout << "Ingrese monto de compra en soles: ";
    cin >> montoCompra;

    cout << "Ingrese dia de la semana (1=Lunes, 2=Martes, ..., 7=Domingo): ";
    cin >> diaSemana;

    // PROCESO
    double porcentajeBase = 0;

    if (montoCompra < 100) {
        porcentajeBase = 0;
    } else if (montoCompra <= 299) {
        porcentajeBase = 10;
    } else if (montoCompra <= 499) {
        porcentajeBase = 20;
    } else {
        porcentajeBase = 30;
    }

    double porcentajeAdicional = 0;

    if (diaSemana == 1 || diaSemana == 2) {
        porcentajeAdicional = 5;
    }

    double porcentajeTotal = porcentajeBase + porcentajeAdicional;
    double descuentoTotal = montoCompra * (porcentajeTotal / 100.0);
    double precioFinal = montoCompra - descuentoTotal;

    // SALIDA
    cout << fixed << setprecision(2);
    cout << "\n==== RESUMEN DE COMPRA =====> endl;
    cout << "Monto original: S/. " << montoCompra << endl;
    cout << "Descuento base: " << porcentajeBase << "%" << endl;
    cout << "Descuento adicional: " << porcentajeAdicional << "%" << endl;
    cout << "Descuento total: S/. " << descuentoTotal << endl;
    cout << "Precio final a pagar: S/. " << precioFinal << endl;

    if (precioFinal > 200) {
        cout << "Pago en 3 cuotas sin intereses disponible!" << endl;
    }

    return 0;
}
```

### Ejercicio 3.1 — Convertidor de unidades para gamers

Código completo y revisado para compilar en Code::Blocks:

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

int main() {
    // ENTRADA
    string nombreJugador;
    double velocidadMs;
    double distanciaMetros;
    double kd;

    cout << "Ingrese nombre del jugador: ";
    getline(cin, nombreJugador);

    cout << "Ingrese velocidad del personaje en m/s: ";
    cin >> velocidadMs;

    cout << "Ingrese distancia del disparo en metros: ";
    cin >> distanciaMetros;

    cout << "Ingrese K/D ratio del jugador: ";
    cin >> kd;

    // PROCESO
    double velocidadKmh = velocidadMs * 3.6;
    double velocidadMph = velocidadMs * 2.237;
    double distanciaPies = distanciaMetros * 3.281;
    double distanciaYardas = distanciaMetros * 1.094;
    double tiempoLlegada = distanciaMetros / velocidadMs;

    string clasificacion;

    if (kd < 1.0) {
        clasificacion = "Novato";
    } else if (kd <= 1.9) {
        clasificacion = "Promedio";
    } else if (kd <= 2.9) {
        clasificacion = "Avanzado";
    } else {
        clasificacion = "Profesional";
    }

    // SALIDA
    cout << fixed << setprecision(2);
    cout << "\n==== ESTADISTICAS INTERNACIONALES =====< endl;
    cout << "Jugador: " << nombreJugador << endl;
    cout << "Velocidad en km/h: " << velocidadKmh << endl;
    cout << "Velocidad en mph: " << velocidadMph << endl;
    cout << "Distancia en pies: " << distanciaPies << endl;
    cout << "Distancia en yardas: " << distanciaYardas << endl;
    cout << "Tiempo en llegar al objetivo: " << tiempoLlegada << " segundos" << endl;

    cout << "\nEl jugador " << nombreJugador << " tiene un K/D de "
        << kd << ", lo que lo clasifica como: " << clasificacion << endl;

    return 0;
}
```

## Ejercicio 3.2 — Calculadora de ahorro para PC Gaming

Código completo y revisado para compilar en Code::Blocks:

```
#include <iostream>
#include <string>
#include <iomanip>
#include <cmath>
using namespace std;

int main() {
    // ENTRADA
    string nombreJugador;
    double presupuesto;
    double ahorroMensual;
    double costoGpu;

    cout << "Ingrese nombre del jugador: ";
    getline(cin, nombreJugador);

    cout << "Ingrese presupuesto total en soles: ";
    cin >> presupuesto;

    cout << "Ingrese ahorro mensual en soles: ";
    cin >> ahorroMensual;

    cout << "Ingrese costo de la GPU en soles: ";
    cin >> costoGpu;

    // PROCESO
    int mesesNecesarios = ceil(presupuesto / ahorroMensual);
    double porcentajeGpu = (costoGpu / presupuesto) * 100.0;
    double dineroRestante = presupuesto - costoGpu;

    // SALIDA
    cout << fixed << setprecision(2);
    cout << "\n==== PLAN DE AHORRO PC GAMING =====> endl;
    cout << "Jugador: " << nombreJugador << endl;
    cout << "Presupuesto total: S/. " << presupuesto << endl;
    cout << "Ahorro mensual: S/. " << ahorroMensual << endl;
    cout << "Costo de GPU: S/. " << costoGpu << endl;
    cout << "Meses necesarios para ahorrar el total: " << mesesNecesarios << endl;
    cout << "Porcentaje que representa la GPU: " << porcentajeGpu << "%" << endl;
    cout << "Dinero restante despues de comprar la GPU: S/. " << dineroRestante << endl;

    if (porcentajeGpu > 50) {
        cout << "La GPU se lleva mas de la mitad del presupuesto." << endl;
    }

    return 0;
}
```

## Ejercicio 4.1 — Sistema de nivel en League of Legends

Código completo y revisado para compilar en Code::Blocks:

```
#include <iostream>
#include <string>
using namespace std;

int calcularNivel(double xpTotal) {
    int nivel = (int)(xpTotal / 500) + 1;

    if (nivel > 18) {
        nivel = 18;
    }

    return nivel;
}

double calcularXpSiguiente(int nivelActual) {
    if (nivelActual == 18) {
        return 0;
    }

    return (nivelActual + 1) * 500;
}

string obtenerTitulo(int nivel) {
    if (nivel >= 1 && nivel <= 5) {
        return "Aprendiz";
    } else if (nivel >= 6 && nivel <= 11) {
        return "Veterano";
    } else if (nivel >= 12 && nivel <= 17) {
        return "Leyenda";
    } else {
        return "Maestro Supremo";
    }
}

int main() {
    // ENTRADA
    string nombreCampeon;
    double xpTotal;

    cout << "Ingrese nombre del campeon: ";
    getline(cin, nombreCampeon);

    cout << "Ingrese XP total acumulada: ";
    cin >> xpTotal;

    // PROCESO
    int nivel = calcularNivel(xpTotal);
    double xpSiguiente = calcularXpSiguiente(nivel);
    string titulo = obtenerTitulo(nivel);

    // SALIDA
    cout << "\n==== SISTEMA DE NIVEL LOL =====> << endl;
    cout << "Campeon " << nombreCampeon
        << ": Nivel " << nivel
        << " - " << titulo
        << " | XP para siguiente nivel: " << xpSiguiente << endl;

    return 0;
}
```

## Ejercicio 4.2 — Calculadora de macros para gym

Código completo y revisado para compilar en Code::Blocks:

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

double calcularProteinasDiarias(double pesoKg, string objetivo) {
    if (objetivo == "Volumen") {
        return pesoKg * 2.2;
    } else if (objetivo == "Definicion") {
        return pesoKg * 2.0;
    } else if (objetivo == "Mantenimiento") {
        return pesoKg * 1.6;
    } else {
        cout << "Objetivo no reconocido. Se usara Mantenimiento." << endl;
        return pesoKg * 1.6;
    }
}

double calcularCalorias(double proteinas, double carbos, double grasas) {
    return (proteinas * 4) + (carbos * 4) + (grasas * 9);
}

int main() {
    // ENTRADA
    string nombre;
    double pesoKg;
    string objetivo;

    cout << "Ingrese nombre del usuario: ";
    getline(cin, nombre);

    cout << "Ingrese peso en kg: ";
    cin >> pesoKg;

    cin.ignore();

    cout << "Ingrese objetivo (Volumen, Definicion, Mantenimiento): ";
    getline(cin, objetivo);

    // PROCESO
    double proteinas = calcularProteinasDiarias(pesoKg, objetivo);
    double carbos = proteinas * 1.5;
    double grasas = proteinas * 0.4;
    double calorias = calcularCalorias(proteinas, carbos, grasas);

    // SALIDA
    cout << fixed << setprecision(1);
    cout << "\n==== CALCULADORA DE MACROS =====" << endl;
    cout << "Usuario: " << nombre << endl;
    cout << "Objetivo: " << objetivo << endl;
    cout << "Proteinas diarias: " << proteinas << " g" << endl;
    cout << "Carbohidratos diarios: " << carbos << " g" << endl;
    cout << "Grasas diarias: " << grasas << " g" << endl;
    cout << "Calorias totales: " << calorias << " kcal" << endl;

    return 0;
}
```

## Ejercicio 5.1 — Intercambio de items en Minecraft (USO DE FOR... \*ADELANTO\*)

Código completo y revisado para compilar en Code::Blocks:

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

void intercambiarItems(string &item1, int &cantidad1, string &item2, int &cantidad2) {
    string tempItem = item1;
    int tempCantidad = cantidad1;

    item1 = item2;
    cantidad1 = cantidad2;

    item2 = tempItem;
    cantidad2 = tempCantidad;
}

void calcularPesoTotal(string items[], int cantidades[], int n, double &pesoTotal) {
    pesoTotal = 0;

    for (int i = 0; i < n; i++) {
        pesoTotal += cantidades[i] * 0.25;
    }
}

int main() {
    // ENTRADA / DATOS DE PRUEBA
    const int n = 4;

    string items[n] = {"Diamante", "Madera", "Hierro", "Piedra"};
    int cantidades[n] = {5, 20, 12, 40};

    double pesoTotal;

    // SALIDA ANTES DEL SWAP
    cout << "==== INVENTARIO ANTES DEL INTERCAMBIO =====< endl;

    for (int i = 0; i < n; i++) {
        cout << i + 1 << ". " << items[i] << " - Cantidad: " << cantidades[i] << endl;
    }

    // PROCESO: intercambio de dos posiciones
    intercambiarItems(items[0], cantidades[0], items[2], cantidades[2]);

    // SALIDA DESPUES DEL SWAP
    cout << "\n==== INVENTARIO DESPUES DEL INTERCAMBIO =====< endl;

    for (int i = 0; i < n; i++) {
        cout << i + 1 << ". " << items[i] << " - Cantidad: " << cantidades[i] << endl;
    }

    calcularPesoTotal(items, cantidades, n, pesoTotal);

    cout << fixed << setprecision(2);
    cout << "\nPeso total del inventario: " << pesoTotal << " kg" << endl;

    return 0;
}
```

## Ejercicio 5.2 — Estadísticas de un torneo de FIFA

Código completo y revisado para compilar en Code::Blocks:

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

void calcularEstadisticas(int golesFavor, int golesContra,
                        double &promGF, double &promGC, int &diferencia) {
    const int partidos = 5;

    promGF = golesFavor / 5.0;
    promGC = golesContra / 5.0;
    diferencia = golesFavor - golesContra;
}

string evaluarRendimiento(int diferencia) {
    if (diferencia > 5) {
        return "Dominante";
    } else if (diferencia >= 1) {
        return "Positivo";
    } else if (diferencia == 0) {
        return "Equilibrado";
    } else {
        return "Critico";
    }
}

int main() {
    // ENTRADA
    string nombreEquipo;
    int golesFavor;
    int golesContra;

    cout << "Ingrese nombre del equipo: ";
    getline(cin, nombreEquipo);

    cout << "Ingrese goles a favor en el torneo: ";
    cin >> golesFavor;

    cout << "Ingrese goles en contra en el torneo: ";
    cin >> golesContra;

    // PROCESO
    double promGF, promGC;
    int diferencia;

    calcularEstadisticas(golesFavor, golesContra, promGF, promGC, diferencia);

    string rendimiento = evaluarRendimiento(diferencia);

    // SALIDA
    cout << fixed << setprecision(2);
    cout << "\n===== ESTADISTICAS DEL TORNEO FIFA =====< endl;
    cout << "Equipo: " << nombreEquipo << endl;
    cout << "Goles a favor: " << golesFavor << endl;
    cout << "Goles en contra: " << golesContra << endl;
    cout << "Promedio de goles a favor: " << promGF << endl;
    cout << "Promedio de goles en contra: " << promGC << endl;
    cout << "Diferencia de goles: " << diferencia << endl;
    cout << "Rendimiento: " << rendimiento << endl;

    return 0;
}
```

## Ejercicio 6.1 — Tienda de skins en Fortnite

Código completo y revisado para compilar en Code::Blocks:

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

int obtenerPrecioVBucks(int rareza) {
    switch (rareza) {
        case 1:
            return 500;
        case 2:
            return 800;
        case 3:
            return 1200;
        case 4:
            return 1500;
        case 5:
            return 2000;
        default:
            return -1;
    }
}

double convertirVBucksASoles(int vbucks) {
    return (vbucks * 34.90) / 1000.0;
}

int main() {
    // ENTRADA
    string nombreItem;
    int rareza;
    int vbucksJugador;

    cout << "Ingrese nombre del item: ";
    getline(cin, nombreItem);

    cout << "Ingrese rareza:" << endl;
    cout << "1. Comun" << endl;
    cout << "2. Poco comun" << endl;
    cout << "3. Raro" << endl;
    cout << "4. Epico" << endl;
    cout << "5. Legendario" << endl;
    cout << "Opcion: ";
    cin >> rareza;

    cout << "Ingrese cantidad de V-Bucks del jugador: ";
    cin >> vbucksJugador;

    // PROCESO
    int precio = obtenerPrecioVBucks(rareza);

    // SALIDA
    cout << fixed << setprecision(2);
    cout << "\n===== TIENDA DE SKINS FORTNITE =====" << endl;

    if (precio == -1) {
        cout << "Item no disponible." << endl;
    } else {
        double precioSoles = convertirVBucksASoles(precio);

        cout << "Item: " << nombreItem << endl;
        cout << "Precio en V-Bucks: " << precio << endl;
        cout << "Equivalente en soles: S/." << precioSoles << endl;

        if (vbucksJugador >= precio) {
            cout << "Puedes comprar " << nombreItem << "! Te quedan "
                << (vbucksJugador - precio) << " V-Bucks." << endl;
        } else {
            cout << "Te faltan " << (precio - vbucksJugador)
                << " V-Bucks para comprar " << nombreItem << "." << endl;
        }
    }

    return 0;
}
```

## Ejercicio 6.2 — Semáforo de aprobación académica

Código completo y revisado para compilar en Code::Blocks:

```
#include <iostream>
#include <string>
using namespace std;

string evaluarEstadoAcademico(double promedio, int creditos) {
    if (promedio >= 15 && creditos >= 16) {
        return "Excelente - Merito academico";
    } else if (promedio >= 13 && creditos >= 12) {
        return "Aprobado - Buen ciclo";
    } else if (promedio >= 11) {
        return "Aprobado - Rendimiento basico";
    } else if (promedio >= 8 && creditos < 12) {
        return "En riesgo - Consejeria academica";
    } else {
        return "Desaprobado - Repetir el ciclo";
    }
}

int main() {
    // ENTRADA
    string nombreAlumno;
    double promedio;
    int creditos;

    cout << "Ingrese nombre del alumno: ";
    getline(cin, nombreAlumno);

    cout << "Ingrese promedio ponderado: ";
    cin >> promedio;

    cout << "Ingrese creditos cursados: ";
    cin >> creditos;

    // PROCESO
    string estado = evaluarEstadoAcademico(promedio, creditos);

    // SALIDA
    cout << "\n==== SEMAFORO ACADEMICO =====> endl;
    cout << "Alumno: " << nombreAlumno << endl;
    cout << "Promedio: " << promedio << endl;
    cout << "Creditos: " << creditos << endl;
    cout << "Estado academico: " << estado << endl;

    if (estado.find("Merito") != string::npos) {
        cout << "Felicitaciones! Tienes beca parcial disponible." << endl;
    }

    return 0;
}
```